Method:
For this problem I decided to use an implementation of Dijkstra's shortest path algorithm as well as a helper function to create an adjacency list from the given times list.

Approach:
This algorithm creates the adjacency list using the aforementioned helper, then uses a priority queue to track nodes and their associated costs. This allows for the algorithm to find the shortest path to each node, and then because the signal propagates to each node in parallel, the algorithm can just take the longest path to travel as the final answer.

Reasoning:
This algorithm is efficient because, as long as the weights are positive Dijkstra will always be able to find the shortest path.

Reflection:
I used these three test cases:
Example 1:
Input: times = [[2,1,1], [2,3,1], [3,4,1]]
n = 4, k = 2
Output: 2

Example 2:
Input: times = [[1,2,1]]
n = 2, k = 1
Output: 1

Example 3:
Input: times = [[1,2,1]]
n = 2, k = 2
Output: -1

Time Complexity:
O((n+e)log n)